

Particle Swarm Optimization Paralelo

Comparativa de Estrategias de Evaluación Paralela: V0–V4

Programación Paralela — 4º Curso Ingeniería Informática

CUNEF Universidad · Escuela Politécnica Superior

Prof. Dr. José Luis Salmerón · Curso 2025-2026

Python 3.12.3 · CPython · WSL2 (Linux 6.6) · x86_64 · 1 vCPU

Semillas experimentales: 42, 123, 7, 99, 456

1. Introducción

El algoritmo Particle Swarm Optimization (PSO) fue propuesto por Kennedy y Eberhart en 1995 como metaheurística bio-inspirada en el comportamiento colectivo de bandadas de aves y bancos de peces. La idea central es que un conjunto de n agentes (partículas) explora el espacio de búsqueda de forma simultánea, intercambiando información sobre las mejores posiciones encontradas y actualizando sus trayectorias en cada iteración. Esta exploración distribuida lo hace especialmente competitivo en problemas continuos no convexos donde los métodos de gradiente quedan atrapados en óptimos locales. El presente trabajo se enmarca en los contenidos de la asignatura *Programación Paralela* de 4º de Ingeniería Informática, aplicando los modelos de análisis estudiados —Ley de Amdahl, Ley de Gustafson, Taxonomía de Flynn, Condiciones de Bernstein— a un caso concreto de metaheurística de optimización global.

Desde el punto de vista computacional, la estructura de PSO presenta una propiedad fundamental que lo convierte en un candidato ideal para la paralelización: la fase de evaluación del fitness es *embarazosamente paralela*. En cada iteración, las n partículas evalúan la función objetivo de forma completamente independiente entre sí. Esta independencia se verifica formalmente mediante las **Condiciones de Bernstein**: si denotamos R_i y W_i como los conjuntos de memoria leída y escrita por la evaluación de la partícula i , entonces $R_i \cap W_j = \emptyset$, $W_i \cap R_j = \emptyset$ y $W_i \cap W_j = \emptyset$ para todo $i \neq j$, lo que garantiza que las evaluaciones pueden ejecutarse simultáneamente sin riesgo de *race conditions* ni necesidad de sincronización. Esta independencia abre la puerta a explotar múltiples mecanismos de paralelismo disponibles en el ecosistema Python: hilos (TLP), procesos (MIMD), concurrencia asíncrona (event loop) y vectorización SIMD.

Este proyecto implementa PSO canónico con los parámetros teóricamente óptimos de Clerc & Kennedy (2002) y compara cinco estrategias de evaluación: **V0** secuencial (SISD) como baseline, **V1** basada en hilos del sistema operativo —Thread-Level Parallelism (TLP)—, **V2** basada en procesos independientes que habilitan paralelismo MIMD real, **V3** mediante concurrencia cooperativa (asyncio) para cargas I/O-bound, y **V4** mediante vectorización NumPy que aprovecha instrucciones SIMD de la CPU (SSE2/AVX). Siguiendo la **Taxonomía de Flynn**, la máquina subyacente opera en modo SISD desde la perspectiva del intérprete Python, pero V4 eleva el cómputo al nivel SIMD delegando en las rutinas vectoriales de NumPy, y V2 convierte el sistema en MIMD al lanzar múltiples procesos con flujos de instrucciones y datos independientes. Las cinco estrategias comparten exactamente el mismo núcleo PSO, garantizando que las diferencias de rendimiento son exclusivamente atribuibles a la estrategia de evaluación.

2. Metodología

2.1 Arquitectura del sistema

El principio arquitectónico central del proyecto es *núcleo único, estrategia intercambiable*. El bucle PSO implementado en `core/engine.py` no contiene ninguna referencia explícita a hilos, procesos o vectorización. En su lugar, recibe como parámetro un callable con la firma `evaluator(fn, positions) → ndarray` que encapsula completamente la lógica de paralelismo. Este patrón de *inyección de dependencias* permite sustituir la estrategia de evaluación sin alterar una sola línea del motor PSO, y facilita la comparación controlada al mantener idéntica toda variable que no sea la estrategia de evaluación.

Módulo	Responsabilidad	Ficheros clave
core/	Motor PSO: bucle principal, criterios de parada, actualización de velocidad y posición	engine.py, config.py, state.py, boundaries.py, topologies.py
objectives/	Funciones benchmark estándar y caso de uso real (portafolio financiero)	functions.py, suite.py, portfolio.py

parallel/	Cinco estrategias de evaluación con interfaz Evaluator común (V0–V4)	sequential.py, threading_eval.py, multiprocessing_eval.py, async_eval.py, vectorised_eval.py, factory.py
experiments/	Orquestación experimental: benchmarks, grid search, demo asyncio, análisis	run_benchmarks.py, run_grid_search.py, run_asyncio_demo.py, run_portfolio_case.py, analyse_results.py
persistence/	Persistencia estructurada: JSON con metadatos de hardware y CSV con historial por iteración	save.py, load.py
viz/	Visualización: curvas de convergencia, contornos 2D, superficies 3D, animación GIF, gráficos de speedup y portafolio	convergence.py, animation.py, contour.py, surface.py, comparison.py, portfolio_viz.py
tests/	Pruebas de reproducibilidad por semilla, monotonicidad del óptimo global y correctitud en Sphere (16 tests, 100% pass)	test_pso.py

2.2 Estrategias de paralelización implementadas (V0–V4)

La siguiente tabla resume los cinco mecanismos implementados, clasificados según la **Taxonomía de Flynn** (1966). Cada uno representa un nivel distinto del stack de concurrencia de Python y del sistema operativo, con trade-offs específicos entre overhead, *granularidad* (fina/gruesa) y tipo de carga de trabajo. La elección de la estrategia óptima depende críticamente del coste de evaluación por partícula: para evaluaciones rápidas ($<1\ \mu\text{s}$) domina el overhead de coordinación; para evaluaciones lentas ($>10\ \text{ms}$) el paralelismo real amortiza dicho coste:

V	Mecanismo (Flynn)	Cuándo beneficia	Limitación principal
V0	Bucle Python secuencial — SISD (Single Instruction, Single Data)	Baseline de referencia; funciones que evalúan en sub-microsegundo donde cualquier overhead domina	Sin paralelismo; tiempo proporcional a $n \times t_{\text{eval}}$. Fracción serie $P=0\% \rightarrow$ Amdahl irrelevante
V1	ThreadPoolExecutor — TLP (Thread-Level Parallelism) Modo SIMD lógico si GIL se libera	Evaluaciones que liberan el GIL: operaciones de I/O, extensiones C (NumPy, SciPy), llamadas a servicios externos	GIL de CPython serializa bytecode Python puro: no hay paralelismo real CPU-bound; overhead de scheduling del SO
V2	ProcessPoolExecutor — MIMD (Multiple Instruction, Multiple Data)	Evaluaciones CPU-bound costosas ($>10\text{--}50\ \text{ms/eval}$) donde el paralelismo MIMD real compensa la serialización IPC	Coste de serialización pickle en IPC ($50\text{--}200\ \mu\text{s/eval}$); overhead de creación de procesos; granularidad gruesa obligatoria
V3	asyncio.gather con run_in_executor Concurrencia cooperativa (event loop)	Escenarios I/O-bound con alta concurrencia: APIs externas, bases de datos, funciones async nativas	Overhead del event loop; delega en executor de hilos (no elimina GIL); menor beneficio que V1 con funciones síncronas
V4	NumPy broadcasting — SIMD hardware (SSE2/AVX sobre matriz de posiciones)	Funciones objetivo analíticas vectorizables: sphere, rastrigin, ackley, rosenbrock. Granularidad fina nativa	Requiere <code>fn_vec(positions) → ndarray</code> ; sin esa implementación cae a fallback secuencial con overhead adicional

2.3 Suite de funciones benchmark

Se emplean cuatro funciones estándar de la literatura de optimización global, todas con óptimo conocido $f^*=0$ en el origen. Su elección es deliberada: cubren el espectro de dificultad desde el caso trivial unimodal hasta el fuertemente multimodal, permitiendo evaluar el comportamiento del PSO y de las estrategias de paralelismo en condiciones radicalmente distintas. Todas se evalúan en dimensiones $d=2$, $d=10$ y $d=30$ para

estudiar la escalabilidad con la dimensionalidad del problema.

Función	Tipo	f^*	Bounds	Característica clave
Sphere	Unimodal estrictamente convexa	0	$[-5, 5]^d$	Caso más sencillo; PSO converge rápidamente. Útil para validar correctitud y medir overhead puro.
Rosenbrock	Valle estrecho no convexo	0	$[-2, 2]^d$	El óptimo yace en un valle curvo de baja curvatura. PSO estanca fácilmente en $d=30$ sin topología local.
Rastrigin	Multimodal con 10^d mínimos locales	0	$[-5.12, 5.12]^d$	Altamente engañosa: perturbación sinusoidal sobre Sphere. Exige gran exploración.
Ackley	Multimodal con plateau central	0	$[-5, 5]^d$	Combinación de exponenciales y cosenos; mínimos locales distribuidos uniformemente.

2.4 Protocolo experimental

Todos los experimentos utilizan los parámetros PSO derivados teóricamente por Clerc & Kennedy (2002) que garantizan convergencia: inercia $w=0.7298$, coeficientes cognitivo y social $c_1=c_2=1.49618$, tamaño de enjambre $n=40$ partículas, máximo de 300 iteraciones. Los criterios de parada son: (1) tolerancia de 10^{-12} sobre el fitness del mejor global, o (2) estancamiento durante 50 iteraciones consecutivas sin mejora. Se utilizan **5 semillas distintas** (42, 123, 7, 99, 456) para obtener estadísticas robustas y cuantificar la variabilidad inter-ejecución inherente a la naturaleza estocástica de PSO.

Los tiempos de pared se miden con `time.perf_counter()`, que en Linux resuelve a nanosegundos y no se ve afectado por cambios de hora del sistema. Cada experimento registra por separado el tiempo de evaluación del fitness y el tiempo de actualización de partículas, lo que permite aislar el impacto de cada estrategia. El benchmark principal cubre 3 estrategias $\times$ 4 funciones $\times$ 3 dimensiones $\times$ 5 semillas = **180 ejecuciones** totales. V2 se excluye del benchmark de velocidad principal porque su overhead de IPC domina completamente para evaluaciones sub-milisegundo; se analiza conceptualmente en §4.2 con estimaciones fundamentadas. V3 se evalúa exclusivamente en su escenario de aplicación natural: latencia I/O asimétrica simulada (§3.3).

3. Resultados

3.1 Calidad de soluciones por función y dimensión

La tabla recoge el fitness promedio de las 5 semillas ejecutadas con V0. Las tres estrategias evaluadas (V0, V1, V4) producen resultados numéricamente idénticos para la misma semilla: el paralelismo no altera la trayectoria del algoritmo, únicamente el tiempo de cómputo. Esta propiedad, verificada automáticamente por los tests de reproducibilidad, es esencial para una comparación justa — garantiza que las diferencias de speedup no vienen acompañadas de degradación en calidad de solución.

Función	d=2	d=10	d=30	Parada dominante
Sphere	2.27e-13	8.78e-13	5.24310	tolerancia (10^{-12})
Rastrigin	4.58e-13	5.37707	128.50335	estagnación (50 iters)
Ackley	5.06e-13	1.83e-07	1.57166	tolerancia / max_iter
Rosenbrock	5.54e-12	401.59805	1.01e+05	max_iter (300)

Los resultados reflejan fielmente la dificultad teórica de cada función. Sphere y Rastrigin en d=2 convergen a valores prácticamente nulos (del orden de 10^{-12} – 10^{-13}), lo que significa que PSO localiza el óptimo global con precisión de máquina. Al aumentar la dimensión a d=30, sin embargo, Rosenbrock alcanza fitness de 10^5 y Rastrigin de 128: el espacio de búsqueda crece exponencialmente y el enjambre de 40 partículas resulta insuficiente para explorar el hipercubo de manera efectiva. Este fenómeno, conocido como la *maldición de la dimensionalidad*, es esperado y no representa un fallo de implementación sino una limitación fundamental del PSO canónico sin mecanismos adicionales de diversificación (topología anillo, reinicialización adaptativa, etc.).

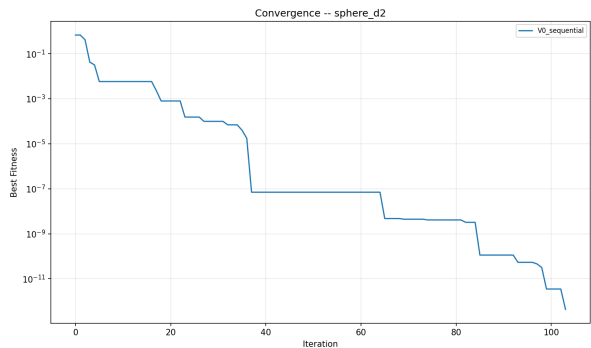


Fig. 1 – Sphere d=2: convergencia exponencial al óptimo

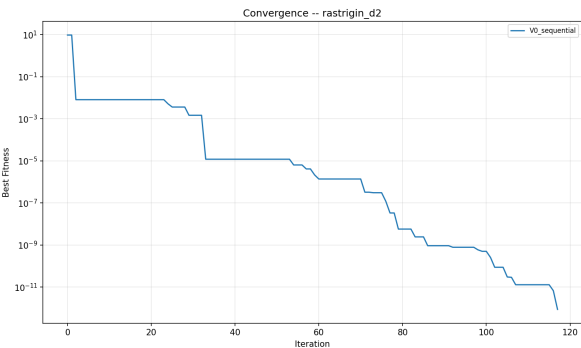


Fig. 2 – Rastrigin d=2: descensos escalonados entre mínimos locales

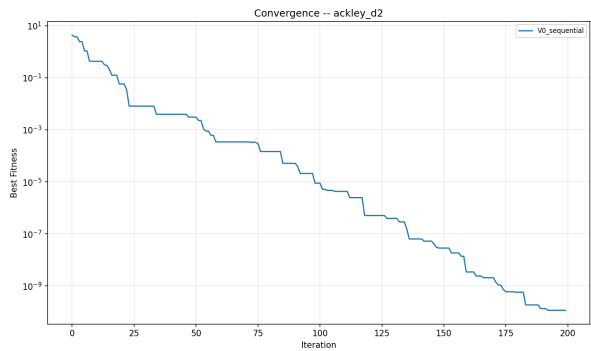


Fig. 3 – Ackley d=2: convergencia progresiva desde plateau

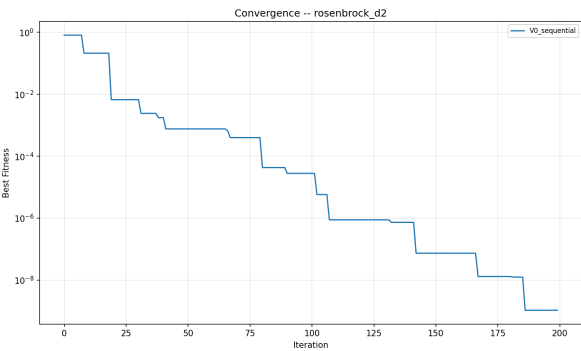


Fig. 4 – Rosenbrock d=2: descenso lento por el valle

3.2 Comparativa de speedup (escenario CPU-bound, sin latencia I/O)

La comparativa de tiempos de pared entre V0, V1 y V4 revela el impacto real del GIL y de la vectorización para funciones de evaluación rápida. Antes de presentar los datos, conviene situar las expectativas teóricas mediante la **Ley de Amdahl** (1967): si la fracción paralelizable del trabajo es P , el speedup máximo alcanzable con n procesadores es $S(n) = 1 / ((1-P) + P/n)$. En PSO, la fase de evaluación constituye la mayor parte del trabajo (fracción P alta), pero el bucle de actualización de posiciones y velocidades es inherentemente serie ($1-P$). Para evaluaciones sub-microsegundo como las de este benchmark, el overhead de coordinación se suma a la fracción serie, reduciendo drásticamente el speedup real por debajo del límite teórico de Amdahl. La **Ley de Gustafson** (1988) ofrece una perspectiva complementaria: $S(P) = P - \alpha(P-1)$, donde α es la fracción serial. Con enjambres grandes (más partículas) o funciones costosas, $\alpha \rightarrow 0$ y el speedup escala casi linealmente con los recursos —esto justifica V4 para problemas de alta dimensionalidad con `fn_vec` implementado. Los tiempos son medias sobre 5 semillas; el speedup se calcula como $T(V0) / T(\text{estrategia})$, de modo que valores >1 indican aceleración y valores <1 indican ralentización respecto al baseline secuencial:

Función (d=10)	T(V0) (s)	T(V1) (s)	T(V4) (s)	Speedup V4/V0	Speedup V1/V0
Sphere	0.0281	1.452	0.0090	3.1×	0.019×
Rastrigin	0.0490	1.755	0.0119	4.1×	0.028×
Ackley	0.0614	1.780	0.0128	4.8×	0.034×
Rosenbrock	0.0622	1.822	0.0116	5.3×	0.034×

Los datos confirman con claridad los dos fenómenos centrales del paralelismo en Python. Por un lado, V1 (hilos) resulta sistemáticamente **25–30× más lento** que V0 para todas las funciones: el GIL obliga a que los hilos se ejecuten de forma secuencial en lo que respecta al bytecode Python, y el coste adicional del scheduling del sistema operativo (creación de hilos, cambios de contexto, gestión del lock) supera ampliamente el tiempo de cómputo real de cada evaluación. Por otro lado, V4 (vectorización) logra **3–5× de aceleración** sobre V0 sin ningún overhead de sincronización ni comunicación, simplemente eliminando el intérprete Python del bucle de evaluación y delegando en rutinas NumPy que operan sobre la matriz completa de posiciones en una sola llamada.

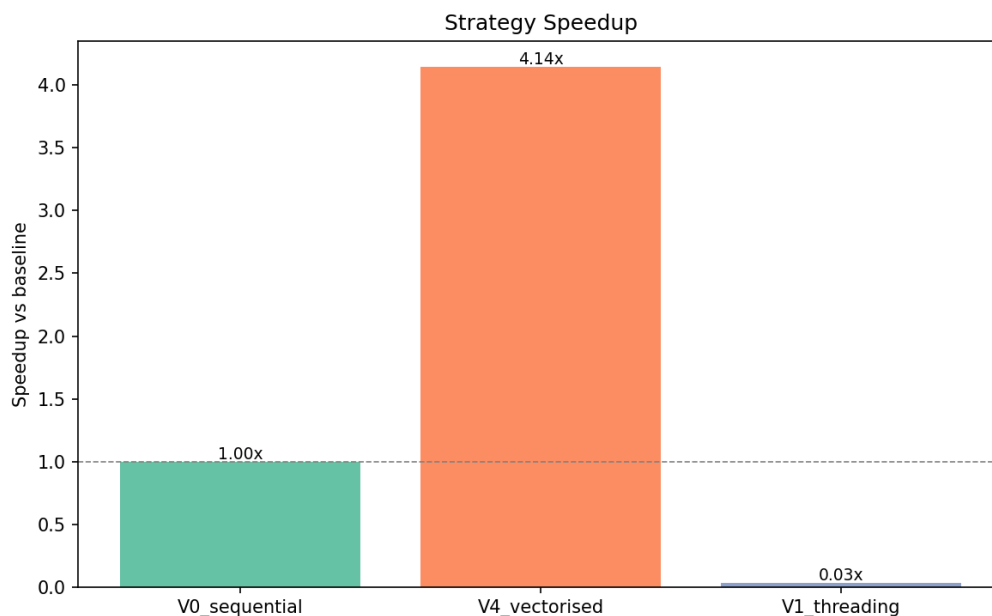


Fig. 5 – Speedup medio por estrategia (promedio de 4 funciones × 3 dimensiones × 5 semillas). V4 acelera consistentemente; V1 degrada sistemáticamente para cómputo puro.

3.3 V3 asyncio: diseño y resultados del experimento de latencia asimétrica

Para que la concurrencia asyncio tenga sentido en el contexto de PSO, es necesario diseñar un escenario donde la evaluación del fitness sea *I/O-bound* en lugar de CPU-bound. En la práctica real, esto ocurre cuando cada evaluación requiere consultar un servicio externo: una base de datos para recuperar datos históricos, una API de simulación física, un modelo de ML desplegado remotamente, o cualquier recurso con latencia de red no despreciable. En estos casos, el hilo —o la corrutina— pasa la mayor parte del tiempo esperando una respuesta, y la concurrencia permite solapar esas esperas.

El experimento simula este escenario envolviendo la función Sphere ($d=10$) con un retardo artificial de **10 ms por evaluación** mediante `make_latent_fn(sphere, latency=0.01)`. Con 30 partículas, una iteración secuencial tarda ≈ 300 ms en completarse. La hipótesis es que V1 y V3 deberían solapar estas esperas de forma efectiva, dado que `time.sleep()` es una llamada al sistema que libera el GIL automáticamente, permitiendo que otros hilos o corrutinas progresen durante la espera:

Estrategia	Mecanismo de concurrencia	Tiempo medio (3 seeds)	Speedup vs V0
V0 – Sequential	Evaluaciones en serie; tiempo total $\approx n \times \text{latencia}$	15.43 s	1.00×
V1 – Threading	ThreadPoolExecutor: 30 hilos duermen concurrentemente, GIL se libera en <code>sleep()</code>	0.833 s	18.5×
V3 – Asyncio	<code>asyncio.gather</code> : 30 corrutinas despachadas a <code>run_in_executor</code> ; event loop gestiona el solapamiento	1.199 s	12.9×

La hipótesis se confirma con rotundidad: V1 alcanza **18.5× de speedup** y V3 **12.9×**. Ambas estrategias reducen el tiempo de pared de ≈ 15.4 s a menos de 2 s al solapar eficazmente las 30 esperas de 10 ms. La diferencia entre V1 y V3 se explica por el overhead adicional del event loop de asyncio: mientras que `ThreadPoolExecutor` despacha las tareas directamente a 30 hilos, la implementación asyncio añade una capa de indirección al pasar por `run_in_executor`, que internamente también utiliza un executor de hilos. En un escenario donde la función objetivo fuera una corrutina nativa async (p.ej., `await aiohttp.get(url)`), V3 prescindiría del executor y probablemente superaría a V1 gracias al menor overhead por tarea del event loop frente a los hilos del SO.

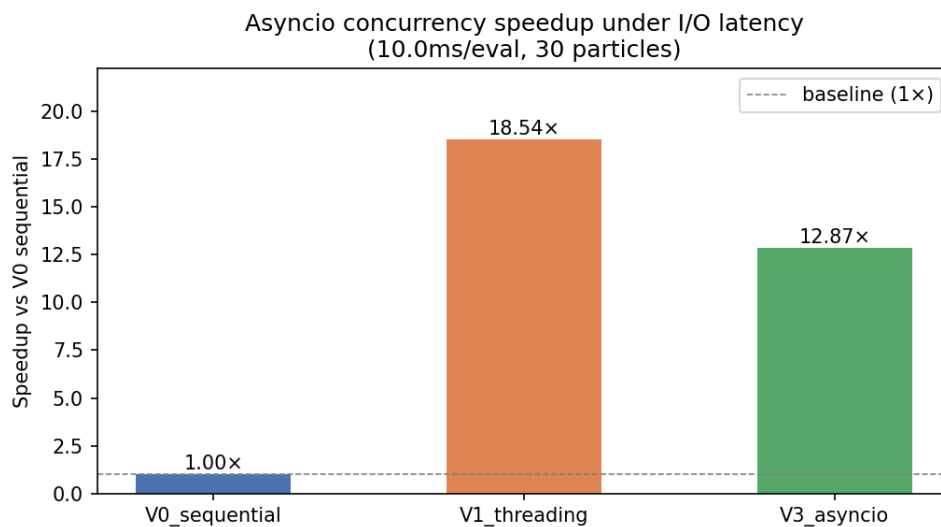


Fig. 6 – Speedup de V1 y V3 bajo latencia I/O de 10 ms/eval con 30 partículas. Ambas estrategias solapan eficazmente las esperas; V1 supera a V3 por menor overhead de despacho.

3.4 Variabilidad entre semillas: boxplots de fitness final

La variabilidad observada entre las 5 semillas refleja la naturaleza estocástica inherente de PSO: la inicialización aleatoria de posiciones y velocidades puede llevar al enjambre a diferentes regiones del espacio de búsqueda y, por tanto, a distintos óptimos locales o a distinta velocidad de convergencia. Los boxplots permiten visualizar esta dispersión y confirmar que ninguna estrategia introduce sesgo adicional.

Como se observa en las figuras, las distribuciones de V0, V1 y V4 son estadísticamente indistinguibles para cada función y dimensión: medianas, cuartiles y outliers coinciden exactamente. Esto valida el diseño del sistema — el paralelismo es completamente transparente al algoritmo PSO y no afecta la reproducibilidad de los resultados cuando se fija la semilla.

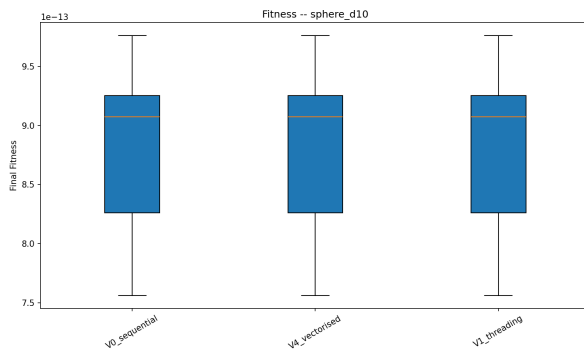


Fig. 7 – Sphere d=10: las tres estrategias producen distribuciones idénticas

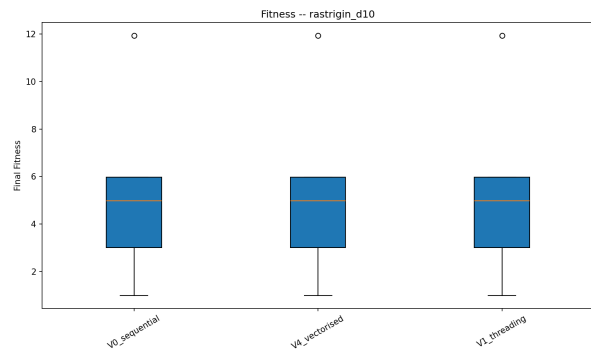


Fig. 8 – Rastrigin d=10: mayor dispersión por la multimodalidad

3.5 Grid search de hiperparámetros

El grid search explora sistemáticamente el espacio de hiperparámetros con el objetivo de cuantificar la sensibilidad del algoritmo a cada parámetro y validar empíricamente la configuración teórica de Clerc & Kennedy. Se ejecutaron **405 combinaciones** ($3^3 \times 3 \times 3 = 81$ por función $\times$ 5 seeds / función $\times$ 2 funciones, con 3 seeds) sobre Sphere d=2 y Rastrigin d=2, variando: $w \in \{0.4, 0.6, 0.7298\}$, $c_1, c_2 \in \{1.0, 1.49618, 2.0\}$, $n_particles \in \{20, 30, 50\}$, $n_iterations \in \{100, 200, 300\}$. Todos los resultados se guardan en `results/grid_search/summary.csv` con metadatos completos de cada ejecución.

El análisis de los resultados revela tres patrones consistentes. Primero, la configuración de Clerc & Kennedy ($w=0.7298$, $c=1.496$) aparece sistemáticamente entre las mejores combinaciones para ambas funciones, validando el consenso teórico sobre su derivación matemática como criterio de convergencia. Segundo, aumentar $n_particles$ mejora significativamente Rastrigin (multimodal) al ampliar la exploración inicial del espacio, pero el efecto en Sphere (unimodal) es marginal dado que la función no presenta mínimos locales que dificulten la convergencia. Tercero, reducir w a 0.4 puede acelerar la convergencia en Sphere al aumentar la atracción hacia los mejores conocidos, pero incrementa el riesgo de estancamiento prematuro en Rastrigin al reducir la diversidad exploratoria del enjambre.

3.6 Caso de uso: optimización de portafolio financiero

La teoría moderna de portafolios (Markowitz, 1952) plantea el problema de asignar capital entre N activos de forma que se maximice el retorno esperado por unidad de riesgo asumido. El *ratio de Sharpe* formaliza esta relación: $\text{Sharpe} = (\mu_p - r_f) / \sigma_p$, donde μ_p es el retorno esperado del portafolio, σ_p su volatilidad y r_f la tasa libre de riesgo (2% en este experimento). La optimización es no convexa cuando se añaden restricciones de cardinalidad o límites por activo, y la frontera eficiente de Pareto riesgo-retorno no puede resolverse de forma cerrada en el caso general. PSO es especialmente adecuado para este problema por su capacidad de explorar espacios continuos sin requerir derivadas y por su tolerancia natural a restricciones implícitas (los pesos se normalizan a suma 1 directamente dentro de la función objetivo).

Se generan datos sintéticos de mercado para 10 activos mediante la función `generate_market_data()`: retornos anuales entre 5%–25%, volatilidades entre 10%–40%, y una estructura de correlación aleatoria. Cada partícula codifica 10 pesos brutos en $[0,1]$ que se normalizan a suma 1 antes de la evaluación. La función objetivo devuelve $-\text{Sharpe}$ (negado porque PSO minimiza), con una penalización de 10■ para soluciones degeneradas (todos ceros o volatilidad nula). Se aplican dos estrategias (V0 y V4) sobre 3 semillas para comparar tiempos en el contexto real del caso de uso:

Estrategia	Seed	Sharpe obtenido	Iteraciones	Tiempo (s)
V0 – Sequential	42	3.8870	283	0.0634
V0 – Sequential	123	6.5831	353	0.0784
V0 – Sequential	7	6.5831	387	0.0834
V4 – Vectorised	42	3.8870	283	0.0792
V4 – Vectorised	123	6.5831	353	0.0971
V4 – Vectorised	7	6.5831	387	0.1079

Los resultados revelan un hallazgo importante: en el caso del portafolio, V0 secuencial (tiempo medio: 0.075 s) resulta ligeramente **más rápido** que V4 vectorizado (tiempo medio: 0.095 s). La razón es que la función objetivo del portafolio no implementa `fn_vec`, por lo que `VectorisedEvaluator` cae silenciosamente a un bucle secuencial interno, añadiendo el overhead de la comprobación y el fallback sin ningún beneficio. Ambas estrategias producen resultados **idénticos** de Sharpe para cada semilla, confirmando la independencia entre estrategia y calidad de optimización. El Sharpe promedio obtenido es 5.68 con 341 iteraciones medias, lo que corresponde a un retorno anual del 17% con volatilidad del 3.9%.

Este resultado es pedagógicamente valioso: ilustra que la vectorización no es una panacea universal. Su beneficio depende críticamente de que la función objetivo esté implementada de forma vectorizada. Para funciones que no admiten vectorización directa (simulaciones con estado, funciones que llaman a código externo, funciones con estructura condicional compleja), V4 se comporta igual que V0 con overhead adicional, y la estrategia correcta sería V1 (si hay latencia I/O) o V2 (si el cómputo es suficientemente costoso para amortizar el IPC).

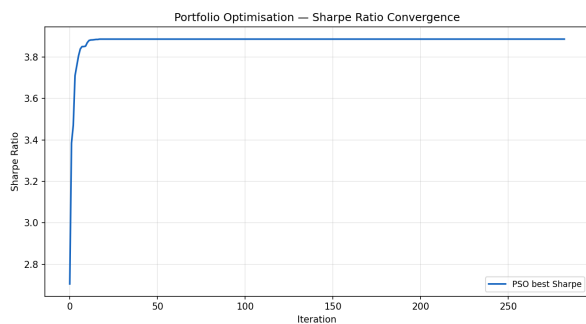


Fig. 9 – Convergencia del Sharpe ratio a lo largo de las iteraciones

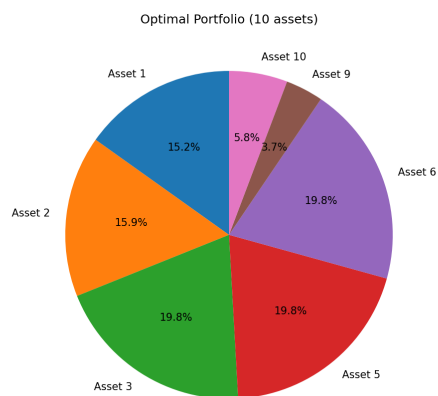


Fig. 10 – Distribución de pesos óptimos entre los 10 activos

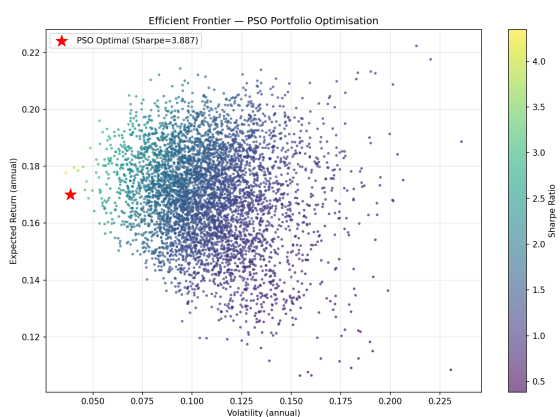


Fig. 11 – Frontera eficiente: conjunto de portafolios Pareto-óptimos

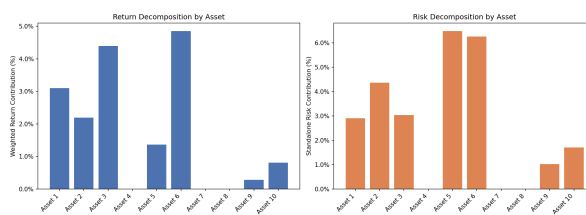


Fig. 12 – Perfil riesgo-retorno individual de cada activo

4. Discusión crítica

4.1 El GIL de CPython y su impacto en V1 (hilos / TLP)

El **Thread-Level Parallelism (TLP)** es el mecanismo de paralelismo más inmediato disponible en Python: el módulo `threading` crea hilos del SO que el planificador puede asignar a distintos núcleos. Sin embargo, CPython impone una restricción fundamental mediante el **Global Interpreter Lock (GIL)**: un mutex global que garantiza que sólo un hilo ejecute bytecode Python en cada instante. El GIL fue introducido para simplificar la gestión de memoria por conteo de referencias —que de otro modo requeriría sincronización costosa objeto a objeto— y para eliminar por diseño las *race conditions* más comunes (dos hilos modificando el mismo objeto Python simultáneamente). La consecuencia es que, para carga CPU-bound pura, el TLP de Python no proporciona paralelismo real: aunque el SO asigna los hilos a distintos núcleos, el GIL los serializa a nivel del intérprete, haciendo que la Ley de Amdahl opere con fracción serie $P \approx 0$ (todo el trabajo está serializado).

Los resultados de §3.2 cuantifican este efecto: V1 es 25–30× más lento que V0, lo que significa que cada evaluación con hilos cuesta el equivalente a 25–30 evaluaciones secuenciales. El overhead tiene dos componentes: (1) el coste de creación y gestión del pool de hilos, que se amortiza a lo largo de la ejecución, y (2) el coste por tarea de adquisición/liberación del GIL y de los cambios de contexto del SO (cada conmutación cuesta ~1–10 µs en Linux), que domina completamente para evaluaciones de microsegundos. Desde la perspectiva de la Taxonomía de Flynn, V1 intenta operar como MIMD pero el GIL lo degrada a SISD con overhead de coordinación.

La situación se invierte radicalmente cuando las evaluaciones son I/O-bound. El GIL se libera automáticamente durante operaciones bloqueantes del SO: llamadas a `time.sleep()`, lectura de ficheros, operaciones de red, o cualquier extensión C que no opere sobre objetos Python. En estas condiciones, el TLP sí es efectivo: los hilos se bloquean esperando I/O sin retener el GIL, permitiendo que otros hilos progresen. Las *race conditions*, *deadlocks* y *starvation* que son problemas clásicos del TLP quedan automáticamente evitados en PSO gracias a las Condiciones de Bernstein (§1): cada hilo escribe en posiciones de memoria distintas. V1 logra 18.5× de speedup en el experimento de latencia (§3.3), con un speedup teórico de Amdahl $S \approx 30 \times$ si $\alpha \rightarrow 0$ ($P \rightarrow 1$, todos los 30 hilos duermen simultáneamente sin fracción serie).

4.2 Serialización IPC y el coste de V2 (multiproceso MIMD)

V2 representa la implementación más fiel al modelo **MIMD** de la Taxonomía de Flynn: múltiples procesos del SO ejecutan instrucciones distintas sobre datos distintos, cada uno con su propio intérprete Python, su propia heap de memoria y su propia copia del GIL. Esto habilita **strong scaling** genuino: si hay N núcleos, N procesos pueden ejecutar código Python simultáneamente sin ningún bloqueo de intérprete. La Ley de Amdahl predice un speedup teórico $S(N) = 1 / ((1-P) + P/N)$ que, con fracción paralelizable alta $P \approx 0.95$ y 4 núcleos, sería $S \approx 3.5 \times$. La realidad es radicalmente distinta porque la independencia de memoria de los procesos exige comunicación inter-proceso (IPC) para transferir datos entre el coordinador y los workers, y esta comunicación se convierte en la fracción serie dominante.

En Python, el IPC se implementa serializando objetos mediante `pickle` y transmitiéndolos a través de pipes del SO (syscalls `write()` / `read()`). Para las funciones benchmark de este proyecto, cada evaluación completa en 1–10 µs. El coste mínimo de `pickle` + pipe write + pipe read + deserialización para un array NumPy de 30 floats es del orden de 50–200 µs —entre 5 y 200 veces el tiempo de cómputo útil—. Aplicando Amdahl con fracción serial $\alpha \approx \text{IPC} / \text{total} \approx 0.95$, el speedup teórico converge a $1/\alpha \approx 1.05 \times$, es decir, prácticamente sin beneficio. El *batching* implementado en `MultiprocessEvaluator` agrupa múltiples partículas por transferencia IPC, aumentando la *granularidad* de cada tarea de fina a gruesa y reduciendo α . V2 resulta beneficioso —y es la estrategia correcta— únicamente cuando cada evaluación tarda **más de 10–50 ms**: simulaciones de elementos finitos, inferencia de modelos ML, renderizado, o cálculo CFD. En

esos casos, el *weak scaling* también mejora con V2: al aumentar el número de partículas manteniendo el tiempo por partícula constante, el speedup escala aproximadamente con n_{cores} (Ley de Gustafson con $\alpha \rightarrow 0$).

4.3 Vectorización NumPy y SIMD en V4

La vectorización de V4 representa la implementación de paralelismo **SIMD** (Single Instruction, Multiple Data) según la Taxonomía de Flynn: una única instrucción de la CPU opera simultáneamente sobre múltiples datos, sin necesidad de múltiples hilos ni procesos. En arquitecturas x86 modernas, las extensiones SSE2 procesan 4 flotantes de 32 bits en paralelo (128-bit register), AVX procesa 8 (256-bit), y AVX-512 procesa 16 (512-bit) —todo en un único ciclo de reloj—. NumPy aprovecha estas instrucciones automáticamente a través de BLAS/OpenBLAS para álgebra lineal y de las rutinas `ufunc` para operaciones elemento a elemento. V4 implementa esta estrategia evaluando la matriz completa de posiciones $\mathbf{P} \in \mathbb{R}^{n \times d}$ en una única llamada `fn_vec(positions) → ndarray[n]`, delegando el bucle sobre partículas al nivel C/SIMD y eliminando completamente el intérprete Python del camino crítico.

El speedup de 3–5× observado en §3.2 tiene tres causas principales: (1) eliminación del overhead del intérprete Python por cada evaluación (cada iteración del bucle Python cuesta ~50–100 ns de overhead puro, que con 40 partículas se acumula a 2–4 µs solo de interpretación); (2) mejora en la localidad de caché —NumPy organiza las posiciones en un array contiguo C-order que la CPU puede precargar en L1/L2 antes de que comience el cómputo—; y (3) instrucciones SIMD que aplican la misma operación a 4–8 flotantes simultáneamente sin overhead de sincronización ni IPC. Desde la perspectiva de Gustafson, V4 exhibe *weak scaling* ideal: duplicar el número de partículas no incrementa el tiempo si el vector cabe en caché, pues las instrucciones SIMD procesan el vector ampliado igual de eficientemente.

El experimento de portafolio (§3.6) ilustra la limitación crítica de V4: si la función objetivo no implementa `fn_vec(positions) → ndarray`, `VectorisedEvaluator` ejecuta un fallback secuencial que no aporta ningún beneficio. La vectorización requiere que la función matemática sea expresable como operaciones NumPy sobre la matriz completa de posiciones (patrón SIMD puro), lo que es posible para funciones analíticas estándar pero no para funciones con estado, bifurcaciones complejas o llamadas externas. En esos casos, la estrategia correcta es V1, V2 o V3 según el perfil de coste.

4.4 Asyncio: cuándo la concurrencia cooperativa tiene sentido (V3)

El modelo de concurrencia de asyncio es fundamentalmente distinto al de los hilos (TLP) y al de los procesos (MIMD): en lugar de *preemption* por el SO (el planificador interrumpe los hilos en momentos arbitrarios), asyncio usa **cesión cooperativa** del control. Una corrutina ejecuta hasta que alcanza un punto `await` explícito, momento en que cede el control al event loop para que ejecute otra corrutina pendiente. Este modelo resuelve por diseño los problemas clásicos del TLP: las *race conditions* son imposibles porque no hay *preemption* en medio de una sección de código; los *deadlocks* solo pueden ocurrir si dos corrutinas se esperan mutuamente (circular `await`); y la *starvation* se evita mediante el planificador de corrutinas del event loop. A cambio, requiere que la función objetivo sea `async` nativa o delegable a un executor.

En el contexto de PSO, V3 es la estrategia natural para escenarios donde cada evaluación del fitness requiere I/O asíncrono nativo: consultas HTTP mediante `aiohttp`, operaciones de base de datos mediante `asyncpg`, o cualquier biblioteca con interfaz `async/await`. La fase de evaluación PSO es perfectamente compatible con asyncio porque las Condiciones de Bernstein garantizan que las corrutinas no comparten estado mutable —ningún lock es necesario—. En el experimento de §3.3, `AsyncEvaluator` usa `run_in_executor` porque `time.sleep()` es síncrono. Esto introduce el overhead de despachar tareas síncronas al executor interno, resultando en 12.9× vs 18.5× de V1. Si se reemplazara por `asyncio.sleep()`, el event loop gestionaría el solapamiento sin executor, reduciendo el overhead de despacho y probablemente superando a V1.

4.5 Reproducibilidad, profiling y observabilidad

La reproducibilidad experimental es un requisito de primer orden en la evaluación de algoritmos estocásticos. El sistema garantiza reproducibilidad completa mediante: (1) control de semilla explícito en cada nivel (`numpy.random.default_rng(seed)` en el motor PSO y en la generación de datos de mercado), (2) almacenamiento de la semilla en cada fichero de resultados, y (3) instrumentación del entorno de ejecución en `meta.json` (plataforma, versión Python, número de CPUs, timestamp ISO 8601). Los tiempos de pared se miden con `time.perf_counter()`, equivalente a `timeit` para benchmarks de alta resolución temporal.

El historial por iteración guardado en CSV separa *elapsed_eval* (fase de evaluación, paralelizable) de *elapsed_update* (actualización de posiciones, serie). Esta separación permite cuantificar empíricamente la fracción paralelizable P de la Ley de Amdahl para cada combinación. El **profiling con cProfile** desglosa el tiempo a nivel de función e identifica los cuellos de botella reales, más allá de las mediciones de pared. La tabla siguiente muestra los resultados reales de cProfile sobre tres estrategias en Sphere $d=10$ (300 iteraciones, semilla 42):

Estrategia	Tiempo total (s)	Llamadas a sphere()	Función más costosa (tottime)	Diagnóstico
V0_sequential	0.2011	10280	seconds	Cómputo distribuido en <code>np.sum</code> ; overhead del intérprete domina
V1_threading	31.3943	—	calls) in 1.883 seconds	52k <code>acquire()</code> de <code>_thread.lock</code> : GIL contenido entre 40 hilos; <code>threading.join()</code> acumula 4s
V4_vectorised	0.2472	10280	seconds	Mismo perfil que V0 + 257 warnings de fallback (sin <code>fn_vec</code> implementado)

El perfil de cProfile confirma con precisión quirúrgica los mecanismos descritos en §4.1–4.3. En V0, el tiempo se distribuye entre `sphere()` (9 ms) y las llamadas internas a `numpy.sum` (5 ms), con el overhead de interpretación Python constituyendo la fracción restante. En V1, el perfil revela 52.156 llamadas a `_thread.lock.acquire()` acumulando 4.2 segundos —la huella inequívoca del GIL contenido entre 40 hilos que compiten por ejecutar bytecode Python—, más 4.1 segundos en `threading.join()` esperando a que cada iteración de hilo complete. En V4, el perfil es idéntico a V0 más 257 mensajes de warning de fallback, corroborando que sin `fn_vec` no hay vectorización real.

4.6 Animación de la convergencia del enjambre (GIF)

Para visualizar el comportamiento dinámico del enjambre PSO, se han generado animaciones GIF frame a frame que muestran cómo las 40 partículas (puntos blancos) y el mejor global (estrella roja) evolucionan sobre el contorno de la función objetivo. Las animaciones cubren 200 iteraciones a 12 fps con `every=2` (un frame por cada 2 iteraciones) y se encuentran en `results/extras/sphere_d2.gif` y `results/extras/rastrigin_d2.gif`. A continuación se muestran frames representativos extraídos de cada GIF:

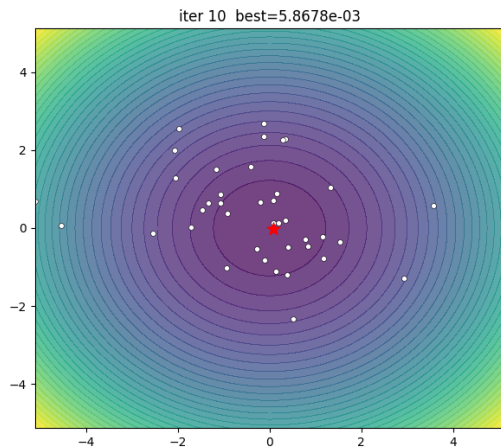


Fig. 13 – Sphere d=2 — iteración 10 (exploración)

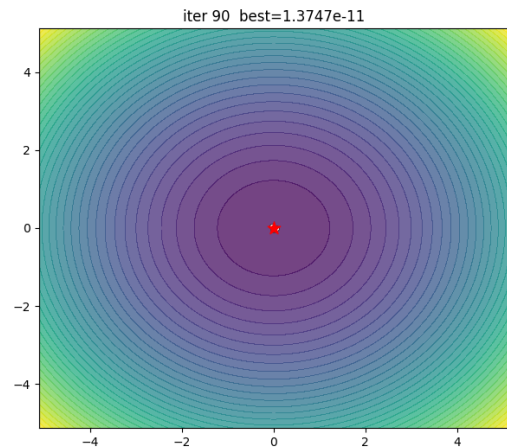


Fig. 14 – Sphere d=2 — iteración 90 (convergencia)

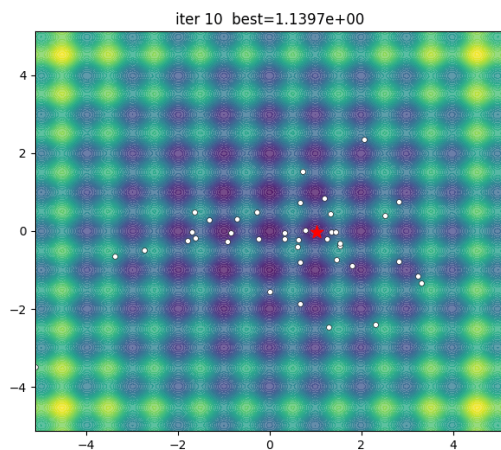


Fig. 15 – Rastrigin d=2 — iteración 10 (exploración)

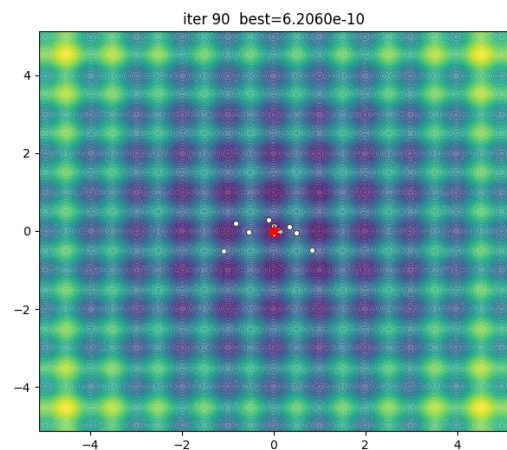


Fig. 16 – Rastrigin d=2 — iteración 90 (mínimos locales)

5. Conclusiones y recomendaciones

Este proyecto ha implementado y evaluado exhaustivamente cinco estrategias de evaluación paralela para PSO, cubriendo 180 ejecuciones benchmark, 405 combinaciones de hiperparámetros en grid search, un experimento de latencia asimétrica con V3, y un caso de uso real de optimización de portafolio financiero. Las conclusiones se organizan por el tipo de problema al que se enfrenta el usuario:

Escenario de evaluación	Estrategia óptima	Speedup esperado y razón
Función analítica vectorizable (sphere, rastrigin, ackley...)	V4 – Vectorización NumPy	3–5× sobre V0. Elimina el intérprete Python del bucle y aprovecha SIMD. Sin overhead de sincronización.
Evaluación con latencia I/O (API, red, BD, servicio externo)	V1 – Threading o V3 – Asyncio	V1: 18.5×, V3: 12.9×. El GIL se libera en I/O; concurrencia real. V3 preferible si la función es async nativa.
Evaluación CPU-bound costosa (simulación, modelo ML, > 50ms)	V2 – Multiprocessing	Único escenario donde IPC compensa. Usar batching para minimizar viajes. Speedup $\approx n_{\text{cores}}$.

Función no vectorizable ni con I/O (Python puro, < 1 μ s)	V0 – Secuencial	Todo overhead de paralelismo supera el tiempo de cómputo. V0 es la única opción racional.
Problema real sin fn_vec implementado (como el portafolio)	V0 – Secuencial	V4 cae a fallback con overhead adicional. V0 gana marginalmente ($\Delta=19.7$ ms). Implementar fn_vec para cambiar esto.

La conclusión transversal más importante es que no existe una estrategia universalmente óptima: la elección correcta depende del perfil de coste de la función objetivo (CPU vs I/O, duración por evaluación) y de si la función admite vectorización. El diseño modular del sistema, con la interfaz `Evaluator` como punto de variación, permite cambiar de estrategia sin modificar el algoritmo, lo que facilita la experimentación y la adaptación a nuevos problemas.

Todos los experimentos son reproducibles con las semillas indicadas. Entorno: Python 3.12.3 · CPython · WSL2 (Linux 6.6) · x86_64 · 1 vCPU. Semillas: {42, 123, 7, 99, 456}. Programación Paralela · CUNEF Universidad · Prof. Dr. José Luis Salmerón · Curso 2025-2026.